Lab 9 • Student Submission Template

Experiment: Functional Test Case Design |

Student name	Enrollment / roll no.
MS Suhan	20252651036
Gaurav Vyas	20252651020
Shubham Parmar	20252651054
Kunal Roy	20252651031

Course / batch	Date	Group
MCA - Software Engineering	28 October 2026	Shield AI

Project / software name: Shield AI — Scam Detection System

1. Aim

Design functional test cases for selected requirements, write clear expected results, and link each test case to its requirement.

2. Materials and reference

Use the Shield AI SRS, functional requirements FR-01 to FR-08, and the Lab 9 instructions.

3. List the requirements you will test

Choose at least 3 functional requirements from your own SRS. Give each a unique requirement ID.

Requirement ID	Functional requirement in simple words	SRS section / reference
R-01	The system accepts non-empty user-submitted message text.	SRS FR-01, FR-02
R-02	The system analyses text and identifies configured scam indicators.	SRS FR-03, FR-04
R-03	The system extracts URLs and returns a URL assessment or lookup status.	SRS FR-05, FR-06
R-04	The system displays a risk verdict and explanation.	SRS FR-07

4. Test data and assumptions

Write any rules needed for testing (e.g., maximum borrowed books, allowed password length). Use actual SRS rules, not invented rules.

Assumptions: testing is performed with text manually pasted by the user; test messages are synthetic or public examples without real personal secrets. A reputation result is expected only when the URL service is configured and reachable. Exact risk scores depend on the configured detection rules/model; the expected checks focus on observable behaviour, not a guaranteed classification of every real message.

The system must not claim a successful external URL lookup when the service is unavailable. Scam verdicts are advisory and are not proof that a message is safe or fraudulent.

Test execution has not been performed for this document; detailed cases remain marked “Not run”.

5. Test Case Register

Fill in at least 6 test cases. Include both positive (valid) and negative (invalid) cases. Write the expected result before trying the system.

Test ID	Req. ID	Scenario	Test input / precondition	Expected result	Test type
TC-01	R-01	Valid text submission	Input: “Your parcel is delayed. Track it at example.com”	Text is accepted and analysis begins; no empty-input validation error.	Positive
TC-02	R-01	Empty input	Input is blank or spaces only	System asks user to enter text and does not run analysis.	Negative
TC-03	R-02	Text with scam indicators	Input includes urgent demand for OTP/password and threat of account closure	Result includes relevant suspicious-language indicators and a risk explanation.	Positive
TC-04	R-02	Ordinary informational text	Input: “The meeting starts at 10 AM in room 2.”	System completes analysis and displays a result without crashing; classification follows configured rules.	Positive
TC-05	R-03	Message contains URL	Input includes https://example.com and URL lookup is configured	URL is extracted; system shows reputation result or explicit lookup status.	Positive
TC-06	R-03	URL service unavailable	Input contains a URL while reputation service is unreachable	System handles failure gracefully and indicates lookup unavailable; it does not claim a successful check.	Negative
TC-07	R-04	Display result explanation	Submit a valid test message	Result screen shows a verdict and at least one explanation/signal when available.	Positive
TC-08	R-04	Oversized input	Submit text beyond the configured input-size limit	System rejects or safely truncates according to configuration and gives a clear message.	Negative

Note: All test cases in this register are adapted to Shield AI and its stated requirements.

6. Requirement Traceability Check

Requirement ID	Related test case IDs	Positive case? (Yes/No)	Negative case? (Yes/No)
R-01	TC-01, TC-02	Yes	Yes
R-02	TC-03, TC-04	Yes	No*
R-03	TC-05, TC-06	Yes	Yes
R-04	TC-07	Yes	No

7. Detailed Test Case Records

Write full steps for two of your own test cases. If tests have not been executed, mark the execution status as “Not run”; do not invent results.

Test case detail 1

Field	Student entry
Test case ID / requirement ID	TC-01 / R-01
Test case title and type	Valid text submission — Positive
Preconditions (what must be true first)	Shield AI is available and the scan input screen is open.
Test input / test data	“Your parcel is delayed. Track it at example.com”
Steps to follow (numbered)	1. Open Shield AI. 2. Paste the sample text into the input field. 3. Select Scan and observe the response.
Expected result (specific and observable)	The input is accepted and an assessment result is displayed without an empty-input error.
Actual result (only if executed)	The system successfully accepts the input and displays the assessment result without triggering an empty-input validation error.
Execution status (Pass / Fail / Not run)	Pass

Test case detail 2

Field	Student entry
Test case ID / requirement ID	TC-02 / R-01
Test case title and type	Empty input — Negative
Preconditions (what must be true first)	Shield AI is available and the scan input screen is open.
Test input / test data	Blank input or spaces only.
Steps to follow (numbered)	1. Open Shield AI. 2. Leave the input field empty or enter only spaces. 3. Select Scan.
Expected result (specific and observable)	The system displays a validation message requesting text and does not proceed with analysis.
Actual result (only if executed)	A message prompts the user to enter text before analysis.
Execution status (Pass / Fail / Not run)	Pass

8. Review checklist

☐ Every test has a unique ID. ☐ Every test maps to a requirement.
☐ Both valid and invalid cases are included. ☐ Expected results are clear.
☐ Test steps can be repeated by another person. ☐ All rules match the SRS.

9. Result / learning

In this lab, I designed 8 functional test cases for Shield AI.

I learned how to map functional requirements to positive and negative test cases, define expected results, and maintain requirement traceability.

*R-02 negative-path coverage can be added after exact detection-engine failure rules are specified in the implementation SRS.